

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

*Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 – Программная инженерия, Системное и
прикладное программное обеспечение*

Дисциплина: Основы профессиональной деятельности

Лабораторная работа №6
Асинхронный обмен данными с ВУ
Вариант 4384

Выполнил:
Карнажицкий Максим Романович
Группа: Р3111

Проверила:
Остапенко Ольга Дмитриевна

г. Санкт-Петербург, 2025

Оглавление

Описание задания.....	3
Текст исходной программы.....	4
Описание программы.....	6
Назначение программы.....	6
Область представления.....	6
Область допустимых значений.....	6
Расположение в памяти.....	6
Методика проверки работоспособности программы (пограничные случаи)....	6
Методика проверки работоспособности программы (общий случай).....	8
Вывод.....	9

Описание задания

По выданному преподавателем варианту разработать и исследовать работу комплекса программ обмена данными в режиме прерывания программы. Основная программа должна изменять содержимое заданной ячейки памяти (X), которое должно быть представлено как знаковое число. Область допустимых значений изменения X должна быть ограничена заданной функцией $F(X)$ и конструктивными особенностями регистра данных ВУ (8-ми битное знаковое представление). Программа обработки прерывания должна выводить на ВУ модифицированное значение X в соответствии с вариантом задания, а также игнорировать все необрабатываемые прерывания.

Введите номер варианта

1. Основная программа должна декрементировать содержимое X (ячейки памяти с адресом $02C_{16}$) в цикле.
2. Обработчик прерывания должен по нажатию кнопки готовности ВУ-3 осуществлять вывод результата вычисления функции $F(X)=2X+4$ на данное ВУ, а по нажатию кнопки готовности ВУ-2 вычесть содержимое РД данного ВУ из X, результат записать в X
3. Если X оказывается вне ОДЗ при выполнении любой операции по его изменению, то необходимо в X записать максимальное по ОДЗ число.

Рисунок 1. Вариант 4384

Текст исходной программы

```

                ORG      0x0

V0:             WORD     $DEFAULT, 0x180
V1:             WORD     $DEFAULT, 0x180
V2:             WORD     $INT2, 0x180
V3:             WORD     $INT3, 0x180
V4:             WORD     $DEFAULT, 0x180
V5:             WORD     $DEFAULT, 0x180
V6:             WORD     $DEFAULT, 0x180
V7:             WORD     $DEFAULT, 0x180

; программа обработки прерывания ВУ-3

INT3:           HLT
                LD       X
                ASL
                ADD      #4
                BNC      IF_POS
                OR       #80                ; если отрицательное, то запоминаем в знаковом
разряде
IF_POS:         OUT      6                ; 2*AC+4 -> OUT3
                LD       X                ; возвращаем исходный X
                IRET

; программа обработки прерывания ВУ-2

INT2:           HLT
                IN       4
                SXTB                ; расширяем знак
                ST       SUB_VAL        ; save IN2 value
                LD       X
                SUB      SUB_VAL        ; AC -= IN2
                HLT
CHECK_DOWN:     CMP      MIN
                BGE      CHECK_UP        ; если больше одз устанавливаем макс
                LD       MAX
                HLT
CHECK_UP:       CMP      MAX
                BLT      ST_ALL_OK
                LD       MAX
                HLT
ST_ALL_OK:     ST       X
```

```

        IRET

; стандартная программа обработки прерываний

DEFAULT:    IRET

        ORG    0x02C

X:        WORD    0xFFBF        ; value X
MAX:      WORD    0x003D        ; max value      = 0000 0000 0011 1101
MIN:      WORD    0xFFBE        ; min value

SUB_VAL:   WORD    ?            ; buffer for IN2 operation

; инициализация векторов прерывания

START:     DI
           CLA
           LD     #0xA
           OUT    5              ; OUT2.MR = 1000|0010 = 1011 = a
           LD     #0xB
           OUT    7              ; OUT3.MR = 1000|0011 = 1100 = b
           JUMP   PROG

; основная программа

PROG:      EI
           CLA
DEC_LOOP:  HLT
           DI                    ; запрет прерываний для атомарности декремента
           LD     X
           CMP    MIN
           BGE    DEC_IF_OK      ; AC >= MIN --> AC--
           LD     MAX            ; AC < MIN  --> AC = MAX
           ST     X
           EI
           JUMP   DEC_LOOP
DEC_IF_OK: DEC
           ST     X
           EI                    ; разрешаем прерывания после декремента
           JUMP   DEC_LOOP

```

Описание программы

Назначение программы

Программа работает с 8-битными знаковыми числами из диапазона ОДЗ (ОДЗ см. ниже). Программа декрементирует значение ячейки X, которое изначально известно. Так как декремент рассматривается как атомарная операция в контексте прерываний, то фрагмент кода отвечающий за декремент ячейки обернут в DI-EI.

Прерывание от ВУ-3 выводит на табло значение $2X+4$. Прерывание от ВУ-2 вычитает из X значение на ВУ-2.

Если вычисления выходят за пределы ОДЗ, то всегда выставляется максимально возможное значение.

Область представления

X – знаковое 16-разрядное число, значащими являются 8 бит (включая знаковый)

Область допустимых значений

$$F(X) = 2X + 4$$

$$- 128 \leq 2X + 4 \leq 127$$

$$- 132 \leq 2X \leq 123$$

$$- 66 \leq X \leq 61,5$$

$$- 66 \leq X \leq 61$$

$$X_{16} \in [FFBE_{16}; 003D_{16}]$$

Расположение в памяти

Вектора прерываний	0x000 - 0x00F
Обработчики прерываний	0x010 - 0x02A
Программа	0x02C - 0x045

Методика проверки работоспособности программы (пограничные случаи)

1. В исходном коде программы заменить NOP на HLT в ячейках: 010, 019, 01F, 023, 027, 040, 045
2. Установить значение X близкое к минимальному по ОДЗ (например FFBF)
3. Загрузить измененную программу в память БЭВМ
4. Запустить программу в режиме РАБОТА с адреса 030
5. Дождаться первого останова: AC = FFBE
6. Продолжить работу программы
7. Продолжить работу программы (да, дажды)
8. Дождаться останова: AC = 003D (мы вышли за минимальное значение ОДЗ)
9. Прожать готовность ВУ-3
10. Продолжить работу программы (дважды)
11. DR ВУ-3 = 0111 1110 – практически максимально возможное число. В рамках функции оно максимально; AC = 003C (декремент)
12. Ввести в ВУ-2 значение: 1111 0000 и прожать кнопку готовности (мы вычитаем отрицательное число из почти максимального из ОДЗ). Записать это число
13. Продолжить работу программы (дважды)
14. Вычесть записанное число из AC = 003C – 1. Новый AC = 004C – в процессе вычисления число превысило ОДЗ. Следующий останов установит корректное значение
15. Продолжить работу программы
16. AC = 003D – установилось максимальное значение, так как мы его превысили.
17. DR ВУ-2 = 0111 1110 – большое положительное число. Прожать кнопку готовности
18. Продолжить работу программы (дважды)
19. AC = FFBF – корректное отрицательное число после вычитания DR ВУ-2.
20. Прожать кнопку готовности ВУ-3 – проверяем работу с отрицательными числами
21. Продолжить работу программы (дважды)

22. Рассчитать АС по формуле $F(X)$. В примере: $2 * BE + 4 = D2$.
Именно это и должны получить в ВУ-3. (1101 0010)

Методика проверки работоспособности программы (общий случай)

1. В исходном коде программы заменить NOP на HLT в ячейках: 010, 019, 01F, 023, 027, 040, 045
2. Установить значение X и записать его
3. Загрузить измененную программу в память БЭВМ
4. Запустить программу в режиме РАБОТА с адреса 030
5. Дважды продолжить работу программы
6. Проверить, что значение в аккумуляторе на один меньше записанного
7. В ВУ-2 записать большое число 1000 0000 (больше ОДЗ)
8. Дважды продолжить работу программы
9. Проверить, что в аккумуляторе лежит 003D – максимальное по ОДЗ число.
10. Установить значение ВУ-2: 0010 0000; записать значение аккумулятора
11. Произвести вычисления $X = (AC) - 20_{16}$. Запомнить значение
12. Дважды продолжить работу программы
13. Проверить, что значение в аккумуляторе совпадает с вычисленным X.
14. Прожать готовность ВУ-3; произвести вычисления $2 * (AC) + 4$
15. Дважды продолжить работу программы
16. Сравнить результат в ВУ-3 с вычисленным.

Вывод

В ходе лабораторной работы я узнал о том, как работать с внешними устройствами в БЭВМ в асинхронном режиме, и об их устройстве. Написал полноценную программу на Ассемблере БЭВМ, а также узнал о различных кодировках.